



Дипломна работа

Тема:

„Разработка на иновативна Linux платформа с изцяло персонализирана архитектура и специализирана среда за ефективна работа“

Разработил: Мартин Светославов Каменов

Направление: код 481 „Компютърни науки“

Професия: код 481020 „Системен програмист“

Специалност: код 4810201 „Системно програмиране“

Ръководител-консултант: Никола Атанасов

Учител по информатика и информационни технологии

гр. Габрово

2025 г.

Съдържание

Съдържание

Увод	4
Теоретично развитие на проекта	5
1. Избор на дистрибуция: Arch Linux	5
2. Мениджмънт на пакети и AUR: Paru	7
3. Графичен сървър: Xorg	8
4. Мениджър на прозорци: i3wm	9
5. Терминален емулятор: Alacritty	12
6. Файлов мениджър: Thunar в минималистична среда	13
7. Статус лента и системна информация: Polybar	14
8. Стартер и превключвател на прозорци: Rofi	15
9. Графичен композитор: Picom	16
10. Управление на звука чрез терминала: Pulsemixer	17
11. Системен мониторинг: Vtop	18
12. Преглед на системна информация: Neofetch	19
13. Текстов редактор: Nano	20
14. Архивиране и разархивиране: File Roller	22
15. Избор на браузър: Google Chrome	23
16. Заснемане на екрана: Gscreenshot	24
17. Преглед и задаване на тапет: Feh като минималистичен инструмент	25
Реализация на проекта	26
1. Подготовка на инсталационен носител	26
2. Стартиране на инсталационната среда	28
3. Инсталиране на Arch Linux	29
4. Първо стартиране и начална конфигурация на системата	31
4.1 Изтегляне на пакети с pacman	31

4.2	Инсталиране на AUR помощник (helper).....	31
4.3	Инсталиране на допълнителни пакети чрез AUR помощник.....	32
5.	Редактиране на файлове, които изискват привилегии.....	32
5.1	Редактиране на pacman.conf	32
5.2	Редактиране на nanoirc.....	33
5.3	Конфигуриране на Limine.....	34
6	Влизане в i3 сесия чрез lu и конфигурация на i3	34
6.1	Местоположение и редакция на i3 конфигурационен файл	35
6.2	Навигация и манипулиране на прозорците	35
6.3	Контроли за функционални приложения	36
7	Конфигурация на терминала Alacritty	37
8.	Теми и шрифтове за GTK приложения	39
9	Настройка на Polybar	39
9.1	Основен конфигурационен файл	39
9.2	Скрипт за стартиране на Polybar	42
10.	Конфигурация на Rofi и други по малки приложения и полезни функции.....	42
10.1	Конфигурация на Rofi.....	42
10.2	Създаване на команда SysMan	43
10.3	Включване на анимации (при наличие на подходящ хардуер).....	43
10.4	Скрипт за смяна на езика	43
10.5	Конфигурация на дребни приложения и елементи:	44
11	Конфигурация на приложения	46
11.1	Настройка на файловия мениджър Thunar.....	46
11.2	Настройка на браузъра Google Chrome	46
	Заклучение	48
	Източници	49

Увод

Операционната система (ОС) е основният софтуерен компонент на всеки компютър, който управлява хардуерните ресурси и координира работата на всички останали програми. Обикновено множество програми работят едновременно, като всяка от тях се нуждае от достъп до централния процесор (CPU), оперативната памет (RAM) и вторичната памет (SSD, HDD и други устройства за съхранение). Операционната система разпределя тези ресурси, осигурявайки стабилност и ефективност при работа.

Съвременните операционни системи използват графичен потребителски интерфейс (GUI), който улеснява потребителя чрез визуални елементи като икони, бутони и менюта. Чрез него взаимодействието със системата става интуитивно, като се използват както графики, така и текст. Най-популярните операционни системи за персонални компютри днес са Microsoft Windows, macOS и Linux. Въпреки сходната функционалност, те се различават съществено по своята архитектура – основно в ядрото, което е сърцето на всяка операционна система. Ядрото е отговорно за управлението на процесите, паметта и хардуера, като осигурява интерфейс между хардуерните ресурси и софтуера, работещ върху тях.

Настоящата дипломна работа има за цел изграждането на иновативна Linux-базирана платформа, чрез внимателен подбор и конфигуриране на дистрибуция, която да отговаря на конкретните изисквания на проекта. Системата се изгражда с фокус върху оптималното управление на потребителските права, ефективното използване на ресурсите и цялостната лекота на работа. В ядрото на графичната среда е интегриран i3 Tiling Window Manager, конфигуриран с персонализирани клавишни комбинации за по-бърз и интуитивен контрол. Осъществена е и цялостна персонализация на работната среда чрез скриптове, които автоматизират поведението на системата и управлението на софтуера. Инсталацията на необходимите пакети се извършва автоматизирано чрез избран помощен инструмент за пакетен мениджмънт.

Финалният етап включва анализ на използването на системните ресурси и сравнение с други операционни системи като Windows, с цел демонстрация на ефективността и оптимизацията на предложеното решение. Очакваният резултат е напълно функционална, персонализирана и лека Linux система, която осигурява

бърза и сигурна работна среда, адаптирана към нуждите на съвременния потребител.

Разработената система може да бъде полезна не само като инструмент за повишаване на продуктивността, но и като демонстрация на умения в областта на системната администрация, скриптиране, оптимизация и конфигурация на Linux среди. Това я прави ценен актив в професионалното портфолио на всеки разработчик. Значението на Linux е неоспоримо – той е основа за по-голямата част от световната ИТ инфраструктура: от сървъри и контейнери, до вградени устройства и облачни решения.

Теоретично развитие на проекта

1. Избор на дистрибуция: Arch Linux

Като основа на системата, която ще бъде изградена, е избрана Arch Linux – дистрибуция, ориентирана към напреднали потребители, която предоставя пълен контрол върху инсталационния процес, системните компоненти и работната среда. Arch се отличава със своя минимализъм – при инсталация не се включват допълнителни програми или графична среда, което позволява да се изгради конфигурацията изцяло според заданието на проекта, без излишен софтуер. Това ниво на гъвкавост е от съществено значение за проекта, чиято основна цел е изграждане на лека, модерна и напълно персонализирана работна среда.

Arch Linux следва модела „rolling-release“, което означава, че системата получава непрекъснати актуализации и винаги разполага с най-новите стабилни версии на ядрото и пакетите. Това се оказва особено полезно при работа със съвременен софтуер и технологии. Все пак, този подход крие известен риск – понякога след актуализация се появява временна несъвместимост между пакети. Въпреки това, Arch се ползва с подкрепата на изключително активна и компетентна общност, а официалната документация (ArchWiki) е една от най-добрите и най-подробни в света на Linux обществото, което значително улеснява справянето с подобни ситуации.[1]

Друг ключов фактор за избора на Arch е наличието на AUR (Arch User Repository) – хранилище, поддържано от общността, в което се публикуват PKGBUILD файлове за софтуер, който не се намира в официалните хранилища. Това позволява да се използват редица малки, но функционално важни програми, създадени от индивидуални разработчици. Например, използваната в този проект

модификация на Picos с поддръжка на анимации не е налична в официалните хранилища и би била трудна за интеграция без AUR. За разлика от други дистрибуции, при които инсталирането на такъв софтуер изисква ръчно изтегляне, компилация и конфигурация, Arch предлага структурирано и удобно решение чрез своята пакетна система.

В процеса на избор на дистрибуция ще се разгледат и други популярни Linux системи. Например, Ubuntu е широко разпространена и много подходяща за начинаещи благодарение на своята лесна инсталация и богат набор от предварително инсталиран софтуер. Тази „готовност за ползване“ обаче се оказва нежелана в случая на проекта – Ubuntu идва със стандартна графична среда и множество допълнителни приложения, които са ненужни и затрудняват изграждането на минималистична система. Освен това, използването на Snap пакети в Ubuntu води до по-бавно стартиране на приложения и усложнява управлението на конфигурации, което прави дистрибуцията по-малко подходяща за проекти с фокус върху производителност и контрол.[2]

Debian, от своя страна, е известен със своята непоклатима стабилност и се използва широко в сървърни среди. За съжаление, тази стабилност идва с цената на използване на по-стари версии на пакетите, което се оказва ограничаващо, когато е необходимо да се работи с нов софтуер или функционалности. Това прави Debian неподходящ за среда, която трябва да бъде лека и съвременна.[3]

Дистрибуцията Fedora е интересна със своята фокусираност върху нови технологии и включването на последни версии на софтуера. Тя предоставя модерна база и сравнително добра поддръжка. Въпреки това, Fedora не позволява същото детайлно ниво на персонализация по време на инсталация, каквото предлага Arch. Освен това, възможностите за лесно инсталиране на неофициален или потребителски софтуер са по-ограничени, което представлява критично ограничение за нуждите на проекта.[4]

Разгледана е и Manjaro – дистрибуция, базирана на Arch, която цели да предложи същата мощ, но с по-лесна инсталация и готова за употреба среда. Макар това да улеснява начинаещите потребители, Manjaro включва множество предварително конфигурирани елементи и пакети, което отнема от контрола върху системата. Също така, актуализациите се забавят спрямо Arch, което премахва част от предимствата на модела „rolling-release“, правейки я по-малко подходяща за среда, изискваща постоянна актуалност.[5]

Gentoo е дистрибуция, която отива още по-далеч в посока гъвкавост – всеки пакет се компилира от изходен код и може да бъде оптимизиран за конкретен хардуер. Това обаче води до много по-бавен и сложен процес на настройка, както и значителни усилия при поддръжка и обновяване. За нуждите на този проект, подобна тежест не се оказва оправдана.[6]

Бе обмислена и възможността да се използва Void Linux – лека и бърза дистрибуция, независима от systemd, която предоставя добра основа за изграждане на минималистична среда. Все пак, Void има по-малка общност и значително по-ограничена документация, което би усложнило процеса на конфигурация и решаване на възникнали проблеми.[7]

С оглед на всички тези фактори, Arch Linux се оказва най-подходящата дистрибуция за основа на работната среда – съчетаваща гъвкавост, минимализъм, отлична документация, достъп до AUR и rolling-release модел, които заедно позволяват изграждането на персонализирана, лека и модерна система, напълно съобразена с изискванията на проекта.

2. Мениджмънт на пакети и AUR: Paru

В процеса на изграждане на персонализирана работна среда на базата на Arch Linux, управлението на допълнителен софтуер, който не се намира в официалните хранилища, е особено важно. Arch Linux разполага с уникално предимство – Arch User Repository (AUR), което предоставя достъп до хиляди потребителски поддържани пакети. Въпреки че `makepkg` позволява ръчното изграждане на пакети от AUR, този подход не е практичен при по-честа употреба. Затова е избран Paru – съвременен помощник за работа с AUR, който автоматизира процесите на търсене, изтегляне, изграждане и инсталиране на неофициални пакети.

Paru е написан на езика Rust и е създаден с фокус върху бързодействие, надеждност и сигурност. Командният му интерфейс наподобява този на `rsync`, което прави прехода между двата инструмента лесен и логичен. Една от основните причини да избира Paru пред други алтернативи е именно тази близост до `rsync`, съчетана с по-висока производителност при работа с AUR пакети. Paru автоматично обработва зависимости, кешира метаданни и при нужда предлага преглед на PKGBUILD скриптовете, което повишава сигурността при инсталиране на по-малко известен софтуер.[8]

В контекста на настоящата система, AUR играе ключова роля – не толкова за достъп до големи, популярни приложения, а по-скоро за инсталирането на малки, специфични инструменти, създадени от независими разработчици. Такива са, например, модифицирани версии на стандартни програми – като модифицирана версия на `Pisom` с поддръжка на анимации, който използвам в конфигурацията си. Този тип софтуер, макар и нишов, е критично важен за изграждане на работна среда с точно определена визия и поведение. Без помощник като `Paru`, достъпът до такъв софтуер би изисквал ръчно изтегляне, компилация и обновяване – процес, който значително би утежнил поддръжката на системата.

Разглеждам и други AUR помощници като `yaу` и `pikaур`, които също са популярни в Arch общността. `Yaу` е написан на Go и е стабилен инструмент с широка потребителска база. Въпреки това, при тестовете, които провеждам, `Paru` се оказва по-бърз и по-лесен за конфигуриране, особено при работа с големи списъци от зависимости. `Pikaур`, от своя страна, използва Python и предоставя удобен текстов интерфейс, но не предлага същото ниво на интеграция с `rasman`, нито същата производителност като `Paru`.^{[9][10]}

Въпреки своите предимства, `Paru` има и своите особености – например, по-малко конфигурационни възможности в сравнение с някои по-стари помощници и необходимостта от по-задълбочено разбиране на процеса при разрешаване на конфликти между зависимости. Това обаче не представлява сериозен недостатък за проекта, тъй като системата, която се изгражда, е добре контролирана и включва внимателно подбран софтуер.

В обобщение, `Paru` е подходящ избор за нуждите на проекта поради своята скорост, ефективност и удобство при работа с AUR. Той позволява лесна и безопасна инсталация и поддръжка на нишов софтуер, без който системата, която се изгражда, не би постигнала търсения функционален и визуален резултат.

3. Графичен сървър: Xorg

В процеса на изграждане на собствена Linux базирана система, изборът на сървър за дисплея е ключово техническо решение, което оказва пряко влияние върху съвместимостта, производителността и стабилността на графичната среда. След внимателно проучване и практически тестове, решавам да използвам Xorg като сървър за дисплея, вместо по-съвременния протокол Wayland. Макар че Wayland е създаден с идеята да подобри архитектурните ограничения на Xorg и наистина предлага по-ниска латентност и вградена поддръжка на композитинг и

анимации, той все още не предоставя пълна съвместимост с всички инструменти, които се използват в конфигурацията.

В системата, се използва специфична модифицирана версия на Picos с поддръжка на анимации, достъпна чрез AUR. Тя осигурява контрол върху визуалните ефекти и плавноста на преходите между прозорци. Макар подобни анимации да са напълно възможни и под Wayland, включително с вградения композитор, текущата конфигурация и избраните от мен инструменти са изградени около Xorg, където поведението и взаимодействието между компонентите са напълно предвидими и стабилни. Изборът не е мотивиран от липса на желание да се работи с по-нова технология, а от нуждата системата да бъде напълно функционална и стабилна от самото начало, с ясни зависимости и поведение.

Друг важен фактор е хардуерната съвместимост. Wayland работи особено добре с AMD графични карти, благодарение на стабилната поддръжка на драйверите, но при други конфигурации - като NVIDIA или по-стари Intel GPU - се наблюдават дребни, но реални проблеми със съвместимостта или производителността. Поради това Xorg остава по-гъвкавото и по-надеждно решение, особено когато системата включва по-специфични компоненти, които изискват утвърдена и предвидима поддръжка.

В дългосрочен план Wayland може да се наложи като стандарт, но за настоящата конфигурация и спецификите на избраните от мен приложения и интерфейси, Xorg остава по-доброто и по-надеждно решение.[11][12]

4. Мениджър на прозорци: i3wm

Една от основните концепции, която определя начина, по който потребителят взаимодейства със своята графична среда, е архитектурата на мениджъра на прозорци – по-конкретно дали той използва stacking (наслагващ) или tiling (подреждащ) подход. Разликата между тези два типа е съществена и оказва влияние както върху потребителското изживяване, така и върху ефективността при работа с множество отворени приложения.

Stacking window мениджърите, които се използват в повечето традиционни десктоп среди като GNOME, KDE и XFCE, позволяват свободно разместване, оразмеряване и наслагване на прозорци един върху друг. Потребителят сам управлява позиционирането и размера на всеки прозорец, като често използва мишка. Това създава визуална гъвкавост, но при работа с повече прозорци често

води до претрупване, припокриване и загуба на ориентация. Управлението чрез мишка и нуждата от чести ръчни намеси също забавя работния процес, особено при по-интензивна употреба на терминали, редактори и системни инструменти.

Tiling window мениджърите, какъвто е i3wm, използват различен подход – всеки нов прозорец автоматично заема определена част от екрана, без да се застъпва с други. Прозорците се подреждат като „плочки“ (tiles) – странично или вертикално – в логическа структура, която потребителят може лесно да пренарежда чрез клавишни комбинации. Този подход води до изключително рационално използване на пространството, премахва нуждата от ръчно позициониране и осигурява постоянна видимост върху всички активни прозорци. Това е особено полезно в среди, където се работи с множество терминали, редактори на код, мониторинг инструменти и други текстови интерфейси.

Причината да се избере tiling мениджър е именно тази ефективност и предвидимост. В контекста на система, изградена за бърза работа, ясна структура и минимално използване на мишка, този подход се оказва най-логичният. Работата чрез клавиатура значително ускорява ежедневните задачи и позволява по-добра концентрация – няма нужда да се мести прозорец или да се търси с мишката коя програма е активна. Освен това, tiling подходът елиминира „визуалния шум“, характерен за stacking средите, където често прозорци остават скрити зад други.

Въпреки че stacking мениджърите предлагат повече визуална гъвкавост и могат да бъдат по-удобни за потребители, които работят основно с графични приложения като браузъри, редактори на изображения или офис пакети, приоритети на проекта са насочени към продуктивност, бърз достъп до инструменти и рационална организация на работното пространство. Затова е избран tiling window мениджър, като i3wm, който отговаря напълно на тези критерии и се вписва идеално в минималистичната и функционално ориентирана структура на системата, която изграждам.

В система се използва i3wm – динамичен тайлинг мениджър на прозорци, който предоставя пълна функционалност, висока продуктивност и възможност за персонализация чрез конфигурация на ниско ниво. Основната причина за избирането на i3wm е контролът, който дава върху подредбата и управлението на прозорците, както и лекотата, се изгражда лека и стабилна графична среда без излишни ресурси и натоварване.

За разлика от традиционните десктоп среди като GNOME или KDE Plasma, които използват наслагващо позициониране на прозорци, i3wm организира всеки нов прозорец в т.нар. тайлинг стил – прозорците се разполагат автоматично един до друг, без да се застъпват. Това значително повишава ефективността при работа с множество приложения, тъй като елиминира нуждата от ръчно местене и оразмеряване. Управлението е почти изцяло базирано на клавишни комбинации, което позволява бърза навигация и фокус върху съдържанието, а не върху управлението на интерфейса.

В процеса на търсене на подходящ тайлинг мениджър на прозорци, се разглеждат и други популярни решения, като bspwm, dwm, AwesomeWM, Xmonad и Sway. Всеки от тях има своите предимства, но и определени ограничения, които не съответстват напълно на нуждите на проекта.

Bspwm, е много лек и предлага чисто разделение между прозоречния мениджър и програма за управление (като sxhkd), което дава голяма свобода, но изисква по-сложна начална конфигурация и поддръжка. В сравнение с него, i3 предлага по-добра документация, по-бързо стартиране и готова логика за клавишни комбинации, което го прави по-подходящ за директно изграждане на функционална среда.

Dwm е друг минималистичен мениджър, но той изисква прекомпилиране при всяка промяна в конфигурацията. Това е подход, който много потребители оценяват заради скоростта и лекотата му, но в приложението на проекта води до излишна усложненост при тестване и настройване. Xmonad има сходни характеристики, но е написан на Haskell, което поставя допълнително изискване да се работи с език за програмиране, само за да се промени базова настройка. За системата това не е оптимално решение.

AwesomeWM е по-функционален и дори включва възможност за изграждане на уиджети, но неговата конфигурация с Lua е по-сложна, а поведението му не винаги е предсказуемо в сравнение с i3. Освен това, за да се постигне максимална стабилност при работа с AwesomeWM, е необходимо значително време за настройка и тестване, което не се налага при i3.

Последният разгледан вариант е Sway – тайлинг мениджър, който се явява "близък" на i3, но работи под Wayland. Въпреки че Sway предлага същия синтаксис за конфигурация и поведение като i3, проекта използва Xorg вместо Wayland което автоматично прави i3 по-естествения избор за тази среда. Sway

работи най-добре с AMD графика и по-нови Wayland-съвместими приложения, докато приложението на системата изисква стабилност с инструменти, които са оптимизирани за Xorg.

Въпреки че i3 има своите ограничения – например липса на вградени анимации, визуални елементи или удобен интерфейс за нови потребители – за цели на проекта той е перфектният избор. Лекотата, с която може да бъде настроен, ясната структура на конфигурационния файл, богатата документация и активната общност го правят подходящ инструмент за изграждане на стабилна и високоефективна работна среда. Именно комбинацията между контрол, предвидимост и функционалност превръща i3 в предпочитания мениджър на прозорци, напълно съвместим с философията на Arch Linux и с тази на проекта.[13]

5. Терминален емулатор: Alacritty

При избора на терминален емулатор за минималистична Linux среда, базирана на Arch Linux и i3wm, Alacritty се откроява като най-подходящ избор благодарение на своята производителност, визуална естетика и отлична съвместимост със съвременни графични среди. Това е терминален емулатор, който използва GPU ускорение чрез OpenGL за рендериране, предоставяйки висока скорост и плавна работа, включително при бързо скролване, визуализиране на големи обеми изходни данни и работа с приложения като btop или neofetch.

Основните предимства на Alacritty са свързани със своята простота и ефективност. За разлика от други емулатори, той не предлага вградена поддръжка на табове или разделяне на прозорци – решение, което е резултат от умишлената философия на проекта. Вместо да се включват комплексни функции, Alacritty разчита на използването на външни инструменти като tmux за управление на сесии, което предлага по-гъвкаво и скриптуемо поведение.

Сравнително разглеждани алтернативи включват Kitty, Terminator, URxvt (rxvt-unicode), st (Simple Terminal) и gnome-terminal. Kitty също използва GPU рендериране и поддържа по-богата функционалност, включително табове и изображения, но има специфичен конфигурационен синтаксис и множество допълнителни функции, които не са необходими в по-изчистени среди. Terminator и gnome-terminal предлагат класически интерфейс с разделяне на прозорци и графични контроли, но са по-тежки, стартират по-бавно и въвеждат зависимости, които не съответстват на минималистичен подход.[14][15]

URxvt и st са насочени към потребители, търсещи възможно най-лек терминален емулатор. Въпреки това, тези решения изискват значителна ръчна конфигурация – често чрез компилация от изходен код – и не предоставят същото ниво на визуална гладкост или хардуерно ускорение, характерни за Alacritty. В st, дори елементарни функции като Unicode поддръжка или цветови теми налагат модификации на сорс кода.[16]

Alacritty използва YAML конфигурационен формат, който е ясен и лесен за редактиране. В комбинация с i3wm, терминалът стартира мигновено и изглежда чисто, като предлага поддръжка на прозрачност, теми, персонализирани шрифтове и хардуерно ускорение. Макар да не поддържа табове по подразбиране, тази функционалност може да бъде добавена чрез инструменти като tmux или zellij, без това да натоварва основния инструмент. Поради своето бързодействие, лекота и модерна архитектура, Alacritty се утвърждава като най-добрият избор за терминален емулатор в контекста на персонализирана, минималистична и високоефективна Linux система.

6. Файлов мениджър: Thunar в минималистична среда

Изборът на файлов мениджър е ключов етап в конфигурирането на персонализирана Linux система, особено когато целта е постигането на баланс между лекота, функционалност и бързина. В контекста на минималистична среда, изградена върху Arch Linux и i3wm, Thunar се утвърждава като подходящ графичен файлов мениджър благодарение на своя интуитивен интерфейс, основни вградени функции и лека интеграция с допълнителни инструменти, без излишно натоварване на системата или усложняване на архитектурата.

Thunar, първоначално разработен за XFCE среда, функционира напълно самостоятелно и е напълно съвместим с всякакви window мениджъри, включително такива с tiling архитектура като i3wm. Сред основните му предимства са бързото зареждане, поддръжката на плъгини, възможностите за drag-and-drop, както и интеграцията с инструменти за работа с архиви, външни носители, скриптове и персонализирани действия. Тези характеристики го правят ефективен избор за ежедневна работа с файлове – включително преглед, копиране, местене, изтриване и разархивиране – без нужда от допълнителни команди или външни програми.

При анализа на алтернативи са разгледани няколко терминални файлови мениджъри, включително ranger, pnn и lf. Те предлагат изключително висока

производителност, управляват се изцяло чрез клавиатура и са в пълно съответствие с минималистичната философия на системата. Въпреки това, при работа с различни видове архиви (zip, tar.gz, 7z, rar и др.), използването им изисква ръчни команди или инсталиране на външни скриптове и плъгини, което затруднява по-бързата работа. Също така, визуалното преглеждане на изображения и съдържание на външни устройства е значително по-удобно в графична среда, каквато Thunar предлага.

Thunar осигурява и поддръжка на контекстно меню с възможност за дефиниране на персонализирани действия, като например отваряне като root или създаване на архив. Не изисква допълнителни демон-процеси, стартира се мигновено и поддържа стабилна работа, което го прави особено подходящ за системи, насочени към бързодействие и чиста конфигурация.

Други алтернативи като PCManFM, Nemo и Dolphin също са обект на разглеждане. Въпреки че предлагат по-разширена функционалност, те са по-тежки и зависими от конкретни графични среди като LXDE, Cinnamon или KDE. В сравнение с тях, Thunar се отличава със своята независимост и лекота, като не налага инсталиране на допълнителни зависимости, което го превръща в логичен избор за използване с i3wm и подобни среди.[17]

В обобщение, Thunar предоставя необходимата функционалност в лек и интуитивен графичен интерфейс, без да компрометира минималистичната концепция на системата. Макар терминалните файлови мениджъри да предлагат определени предимства, необходимостта от визуално ориентирано и бързо управление на файлове и архиви прави Thunar по-подходящ избор за продуктивна ежедневна употреба.

7. Статус лента и системна информация: Polybar

При изграждането на персонализирана и ефективна работна среда в Arch Linux с i3wm, изборът на подходяща статус лента е от съществено значение. След обстоен анализ и практически тестове, Polybar се утвърждава като предпочитан избор поради своята гъвкавост, богата функционалност и възможности за естетическа персонализация.

Polybar представлява бърз и лесен за използване инструмент, който позволява създаване на статус ленти с висока степен на персонализация. Той включва разнообразие от вградени модули, предоставящи информация за основни системни параметри – като дата и час, ниво на батерията, натоварване на

процесора, използване на паметта, мрежова свързаност и други. Това дава възможност за създаване на лента, която отговаря на конкретните нужди и предпочитания на потребителя.

В хода на избора са разгледани и други решения, включително вградената в i3wm i3bar. Макар стабилна и лека, i3bar не предлага същото ниво на гъвкавост и интегрирани модули като Polybar. За показване на информация i3bar разчита на външни инструменти като i3status или i3blocks, докато Polybar предоставя тези възможности нативно, което значително улеснява конфигурацията и поддръжката.

Едно от основните предимства на Polybar е ясната и добре организирана структура на конфигурационния файл, който улеснява добавянето и настройването на модули според предпочитанията на потребителя. Интеграцията с i3wm също е добре реализирана, което осигурява безпроблемно взаимодействие между статус лентата и мениджъра на прозорци.

Въпреки многобройните си предимства, Polybar има и някои особености – например, изисква по-задълбочена първоначална конфигурация в сравнение с по-опростени решения като i3bar. За потребители, търсещи изключителна лекота и минимално използване на ресурси, i3bar може да се окаже по-подходяща алтернатива.[18][19]

Polybar предлага балансирано съчетание от визуална привлекателност, функционалност и персонализация, което го прави предпочитан избор за статус лента в среда, изградена върху i3wm. Макар и да не е най-лекото решение, неговата гъвкавост и възможности за разширяване отговарят на изискванията за прецизен контрол и визуална завършеност на работната среда.

8. Стартер и превключвател на прозорци: Rofi

В процеса на изграждане на персонализирана и ефективна работна среда в Arch Linux с i3wm, изборът на подходящ инструмент за стартиране на приложения и управление на прозорци е от съществено значение. След внимателно проучване и сравнение на наличните опции, Rofi се утвърждава като основен инструмент за изпълнение на тези задачи. Решението се базира на ключови фактори като гъвкавост, богата функционалност и възможности за персонализация, които Rofi предлага.

Rofi представлява мощен и многофункционален инструмент, който комбинира функционалностите на стартер на приложения, превключвател на прозорци и заместител на dmenu. Програмата предоставя широки възможности за

конфигуриране и темизирание, което улеснява адаптацията ѝ спрямо конкретните изисквания на потребителя и дизайна на работната среда. Основен акцент пада върху бързия и интуитивен достъп до приложения и отворени прозорци, което значително повишава продуктивността.[20]

В хода на анализа са разгледани и алтернативи като dmenu – известен с минимализъм и изключително ниско използване на ресурси. Въпреки това, за постигане на функционалности, които Rofi предлага по подразбиране, при dmenu се изискват допълнителни кръпки и ръчни конфигурации. Rofi поддържа разнообразни режими на работа, включително стартиране на SSH сесии, превключване между прозорци и стартиране на команди, без необходимост от външни инструменти. Освен това, поддръжката на теми и персонализацията при Rofi улеснява визуалното интегриране с останалите елементи на средата.[21]

Конфигурационният файл на Rofi е структуриран по ясен и интуитивен начин, което позволява лесно добавяне и настройване на теми и модули. Инструментът поддържа пълна съвместимост с i3wm, като осигурява плавна и безпроблемна работа в рамките на избрания прозоречен мениджър.

Макар и да изисква малко повече ресурси в сравнение с dmenu, Rofi предлага значително повече възможности за персонализация и контрол. За потребители, които търсят изключителна лекота, dmenu може да се окаже по-подходящ, но при системи с акцент върху ефективност и визуална завършеност, Rofi предоставя по-добрия баланс между функционалност и лекота.

Rofi се утвърждава като предпочитан избор за стартиране на приложения и управление на прозорци в среда, базирана на Arch Linux и i3wm. Неговата богата функционалност, гъвкавост и добри възможности за персонализация го правят оптимално решение за изграждане на продуктивна и визуално консистентна работна среда.

9. Графичен композитор: Picom

В изграждането на една цялостна и визуално завършена Linux система, особено когато се използва мениджър на прозорци като i3wm, е необходимо наличието на композитор, който да осигури базови графични ефекти като прозрачност, сенки и плавни преходи между прозорци. За тази цел избирам Picom, който представлява съвременен и активно поддържан форк на по-стария Compton. Основната причина за този избор е неговата лека и стабилна работа, съчетана с възможност за фина персонализация на визуалните ефекти. Picom ми позволява да

създавам по-завършена и естетически приятна среда, без да въвежда излишна сложност или да натоварва системните ресурси по осезаем начин.

Преди да се спра на `Picom`, разглеждам и други композитори като `xcompmgr` и `Compton`. `xcompmgr` е изключително лек, но предоставя само най-базовите ефекти и почти не подлежи на конфигурация. `Compton`, въпреки че предлага по-широка функционалност, вече не се поддържа активно, което е важно съображение за една система, на която разчитам да бъде дългосрочно стабилна и лесна за поддръжка. `Picom` от своя страна обединява предимствата и на двете – той е достатъчно лек, за да бъде напълно съвместим с философията на минималистичната ми `i3` среда, но също така позволява подробна настройка чрез конфигурационен файл, включително възможности за глобална или прозоречно-специфична прозрачност, закръглени ъгли, `fade-in/fade-out` ефекти и др.[22]

Важно е да се отбележи, че композиторите като `Picom` са необходими само при използване на `Xorg`, тъй като при `Wayland` подобен слой за графични ефекти е вграден в самия протокол. Това е още една причина, поради която `Picom` е естественият избор в моята конфигурация. Въпреки че използването на композитор понякога може леко да повиши натоварването на процесора или да създаде артефакти при определени драйвери, в моята система с подходящо избран хардуер и добре конфигуриран драйверен слой такива проблеми не се проявяват.

В заключение, `Picom` предлага точно толкова, колкото е нужно за създаване на една визуално балансирана и отзивчива графична среда, без да бъде тежък или прекалено зависим от външни компоненти. В комбинация с `i3wm`, той осигурява приятен визуален резултат, който повишава комфорта при работа, като същевременно поддържа лекотата и контрола, характерни за цялостната архитектура на системата.

10. Управление на звука чрез терминала: `Pulsemixer`

При изграждането на лека и изцяло клавиатурно ориентирана Linux среда, управлението на звука също трябва да бъде съобразено с цялостната философия на системата. В тази връзка, за аудио контрол е избран `Pulsemixer` – терминален интерфейс за `PulseAudio`, който позволява управление на нивата на звука, изходите и входовете директно от терминала, без нужда от графичен миксер. Това решение се вписва напълно в минималистичната и клавиатурно базирана

структура на система, изградена с i3wm и Alacritty, където целта е да се избегне зависимостта от тежки графични приложения, без да се губи от функционалност.

Преди да бъде избран Pulsemixer, са разгледани и други варианти за аудио контрол, сред които най-разпространени са alsamixer и pavucontrol. Alsamixer е част от ALSA и работи на много ниско ниво, което го прави неподходящ за системи, използващи PulseAudio, без допълнителна конфигурация. Освен това, интерфейсът му не предлага ясно разделение между различните PulseAudio устройства и често се налага превключване през допълнителни команди, за да се управлява звукът коректно. От друга страна, pavucontrol е напълно графично приложение и предлага детайлен контрол върху звуковите устройства и потоци, но използва GTK и вкарва допълнителни зависимости, което го прави по-тежък и не толкова подходящ за среда, ориентирана към бързина и ниско използване на ресурси.[23][24][25]

Pulsemixer предоставя ясен, цветен и функционален интерфейс в терминала, който е напълно достатъчен за ежедневна употреба – включително регулиране на силата на звука, заглушаване на канали и превключване между устройства. Работи бързо, лек е и може да бъде използван дори в remote сесии, което го прави гъвкав инструмент за всеки потребител, който предпочита терминал пред GUI. Макар и да не предлага пълната визуална яснота на pavucontrol, в тази система това не е недостатък, тъй като цялостната концепция е да се използват леки инструменти, които не изискват графична среда, за да функционират пълноценно.

В обобщение, Pulsemixer е логичният избор за система, базирана на Arch и i3wm, където контролът е централен елемент. Той осигурява нужната функционалност, без да прави компромиси с бързодействието или архитектурната чистота на конфигурацията.

11. Системен мониторинг: Vtop

При изграждането на минималистична, но функционално завършена Linux среда, наблюдението на системните ресурси е от съществено значение. За тази цел е избран Vtop – терминален мониторинг инструмент, който предлага интуитивен и визуално структуриран интерфейс за следене на натоварването на процесора, използването на RAM и swp, активните процеси, мрежовата активност и I/O операции. Изборът на Vtop се основава както на неговата висока визуална четливост, така и на добрия баланс между лекота, производителност и функционалност.

Преди избора на Vtop са тествани алтернативи като htop, top и gtop. Top е най-базовият вариант, включен във всяка Linux система, но предоставя ограничен визуален интерфейс и изисква по-задълбочено разбиране на представената информация. Htop предлага по-добра четимост и възможност за интерактивно управление на процеси, но визуалната му структура не е толкова добре организирана за продължително наблюдение. Gtop също е модерен инструмент с графично ориентирано представяне в терминала, но използва Go и към момента има по-слаба поддръжка.[26][27]

Vtop е написан на C++ и е част от серия инструменти, следвайки bashtop и bpytop, но с подобрена производителност и съвместимост. Отличава се с комбинация от полезността на htop и структурирана, цветна визуализация, което улеснява и прави приятно наблюдението на системата, дори при високо натоварване. Интерфейсът е интуитивен, клавишните комбинации са ясно описани, а използваните цветове осигуряват добра видимост, без да намаляват четливостта.[28]

Въпреки че Vtop използва повече ресурси спрямо top или htop, това не се разглежда като недостатък в контекста на съвременен хардуер и добре оптимизирани системи за продуктивна работа. При стартиране в терминал като Alacritty, Vtop зарежда мигновено и осигурява бърз достъп до критична системна информация, което го прави ценен инструмент по време на разработка или диагностика.

В обобщение, Vtop е избран като системен монитор заради своята модерна и добре структурирана визуализация, лесен клавиатурен контрол и достатъчна функционалност, без да излиза извън терминалното приложение. Това го прави подходящо допълнение към минималистична среда, където лекотата и визуалната яснота са приоритет.

12. Преглед на системна информация: Neofetch

В една персонализирана Linux система, изградена с фокус върху минимализъм и естетика, наличието на инструмент, който по ясен и визуално приятен начин представя системната информация, има както практическа, така и представителна стойност. За тази цел е избран Neofetch – лек терминален инструмент, който показва ключови данни за системата в елегантен текстов формат, включително информация за дистрибуцията, ядрото, uptime, RAM,

процесор, резолюция, shell и други, допълнени със стилизирано ASCII лого на използваната операционна система.

Използването на Neofetch е особено удобно при конфигуриране, тестване и споделяне на информация за системата, тъй като с една команда може да бъде визуализиран цялостен системен преглед директно в терминала. Инструментът се вписва идеално в терминално ориентирана среда като тази, базирана на i3wm и Alacritty, като предоставя нужната информация без допълнителна тежест или усложнение.

Алтернативи като screenfetch, ufetch, pfetch и fastfetch също изпълняват сходна роля, предлагайки визуално подобни резултати, но Neofetch се отличава с по-широка поддръжка на системни параметри и по-активна разработка. Докато ufetch и pfetch са по-леки и насочени към крайно минималистични среди, те често показват по-малко данни и предлагат ограничени възможности за персонализация. Fastfetch се развива като модерен алтернативен проект, но към момента разполага с по-малка общност и документация.

Neofetch е лесен за конфигуриране и позволява персонализиране на изхода според конкретни нужди – например скриване или показване само на избрани параметри, добавяне на цветови теми и използване на потребителски ASCII логота. Освен това, инструментът е често използван в скрийншотове и демонстрации на системни конфигурации, което допринася за неговата популярност в Linux общността. Макар и да не увеличава директно производителността, Neofetch се утвърждава като информативен и естетически инструмент, който допринася за усещането за завършеност на системната среда.

Поради своята лекота, визуална привлекателност и възможности за персонализация, Neofetch се вписва отлично в конфигурации, ориентирани към минимализъм и терминална продуктивност. Той допринася за идентичността на системата, като същевременно служи като полезен и визуално приятен инструмент за бърз преглед на системни данни.

13. Текстов редактор: Nano

При изграждането на персонализирана и ефективна работна среда в Linux, изборът на подходящ текстов редактор е от съществено значение. След обстойно проучване и практически тестове, като основен текстов редактор в системата е избран GNU nano. Това решение се основава на няколко ключови фактора,

свързани с лекотата на използване, достъпността и ефективността при изпълнение на ежедневни задачи.

GNU nano е текстов редактор, проектиран с акцент върху простотата и интуитивността. Интерфейсът му е лесен за разбиране, като командите са ясно посочени в долната част на екрана, което улеснява навигацията и използването му дори от потребители с ограничен опит в работата с командния ред. Тази достъпност прави nano особено подходящ за бързи редакции на конфигурационни файлове и други текстови документи.

В процеса на избор са разгледани и други популярни опции като Vim и Emacs. Vim е мощен редактор с богата функционалност, включително поддръжка на макроси, разделяне на прозорци и автоматично довършване на команди. Тези функции го правят изключително ефективен за сложни редакции и програмиране. Въпреки това, стръмната крива на обучение и необходимостта от запаметяване на множество команди могат да представляват предизвикателство за нови потребители. Както се отбелязва в Linux Training Academy, „кривата на обучение на Vim често плаши новите потребители“.

Emacs, от своя страна, представлява не просто текстов редактор, а цяла среда за разработка с вградени функции като уеб браузър, клиент за IRC и дори игри като Tetris. Неговата гъвкавост и възможности за разширение чрез Emacs Lisp го правят мощен инструмент, но сложността на интерфейса и нуждата от овладяване на специфични клавишни комбинации могат да се окажат възпиращ фактор за потребители, търсещи по-опростено решение.

В сравнение с тези алтернативи, GNU nano предлага балансиран подход между функционалност и лесна употреба. Макар да не притежава всички напреднали възможности на Vim и Emacs, nano е напълно достатъчен за повечето ежедневни задачи, свързани с редактиране на текстови файлове. Неговата лекота и интуитивност позволяват на потребителя да се фокусира върху работата си, без да е необходимо предварително задълбочено обучение.

В обобщение, изборът на GNU nano като основен текстов редактор се основава на неговата простота, достъпност и ефективност при изпълнение на ежедневни задачи. Въпреки наличието на по-мощни алтернативи като Vim и Emacs, nano предоставя необходимата функционалност без излишна сложност, което го прави логичен избор за системи, изградени върху принципите на минимализъм и ефективност.[29]

14. Архивиране и разархивиране: File Roller

Въпреки че изгражданата Linux система е силно минималистична и ориентирана към терминална работа, за определени дейности графичните инструменти остават по-подходящ избор. Такъв е случаят при работа с архиви от различни формати, където използването на удобен и интуитивен графичен архиватор е значително по-ефективно от ръчното въвеждане на команди в терминала. Поради това е избран File Roller – лек графичен архиватор, който се интегрира добре със среди като Thunar и предоставя лесен достъп до създаване, преглед и разархивиране на файлове в широк набор от формати, включително .zip, .tar.gz, .7z, .rar и други.

File Roller е част от GNOME екосистемата, но може да бъде използван напълно самостоятелно, без необходимост от зареждане на цялата графична среда. Това го прави подходящ за използване в системи, изградени с лека графична обвивка и мениджър на прозорци като i3, без пълноценен десктоп мениджър. Основното му предимство се състои във възможността с няколко клика да се отваря, преглежда и извлича съдържание от архиви – нещо, което би отнело повече време при използване на терминал, особено при работа с непознати формати или множество архивирани файлове наведнъж.[30]

Сред разглежданите алтернативи са терминалните инструменти unzip, tar, 7z (p7zip) и atool, както и по-леки графични решения като Xarchiver, Engrampa (от MATE) и Ark (от KDE). Въпреки че терминалните приложения предоставят висока степен на контрол, те не са толкова удобни при бърза работа с множество файлове или когато се налага визуално преглеждане на съдържание. Xarchiver е също възможен избор благодарение на своята лекота и независимост, но предлага по-ограничена поддръжка на формати и не толкова изчистен интерфейс. Ark и Engrampa изискват допълнителни зависимости от средите KDE и MATE, което не е желателно в конфигурации, фокусирани върху минимализъм.[31][32][33]

File Roller предлага правилния баланс между функционалност и лекота. Стартира се бързо, поддържа всички нужни формати чрез външни архивни библиотеки и се интегрира ефективно с Thunar. Това осигурява възможност за създаване и извличане на архиви без необходимост от напускане на графичната среда, в която вече се работи с файлове.

В обобщение, File Roller е избран заради своята лекота, надеждност и пълноценна поддръжка на разнообразни архивни формати. Въпреки

съществуването на терминални алтернативи, именно графичният интерфейс и удобството при работа го правят най-подходящото решение за тази част от системата.

15. Избор на браузър: Google Chrome

Въпреки че основната структура на изгражданата Linux система е ориентирана към терминална работа и минимализъм, за основен уеб браузър е избран Google Chrome. Причината е, че той предлага висока степен на съвместимост с уеб технологии, богата екосистема от разширения и теми и е широко разпространен сред потребителите на всички платформи. Макар сред Linux потребителите Mozilla Firefox да е по-често използван браузър – отчасти заради това, че е с отворен код, включен по подразбиране в повечето дистрибуции и с добра репутация за поверителност – Chrome е предпочетен поради своята водеща позиция в световен мащаб и функционалност.

Една от основните причини за този избор е огромният избор от разширения и теми, достъпни чрез официалния уеб магазин на Chrome. Това позволява персонализация на браузъра според конкретните нужди – от блокиране на реклами и проследяване, до инструменти за разработчици, синхронизация между устройства и усъвършенствано управление на раздели. Интеграцията с услугите на Google също е безпроблемна, което е важно при работа с инструменти като Google Drive и Gmail.

Разгледани са и алтернативи като Firefox, Chromium, Brave, Vivaldi и Opera. Firefox, с отворен код и широко използван в Linux общността, показва добра производителност, но при тестове с някои уеб приложения и видеоплатформи Chrome се представя по-добре. Chromium, макар и много близък до Chrome, липсва част от функционалността, свързана със синхронизация и някои затворени компоненти. Brave и Vivaldi предоставят допълнителни функции като защита срещу проследяване и панелни разширения, но внасят допълнителна сложност, която не съответства на принципите на минимализъм, следвани при изграждането на системата.

Google Chrome се интегрира отлично с избраната графична среда. Въпреки че не е най-лекият браузър, неговата стабилност, възможности за разширения и пълна съвместимост със съвременните уеб стандарти го правят практичен избор. За разлика от по-леките алтернативи, Chrome гарантира, че всички уебсайтове и

приложения функционират безпроблемно – дори в система, където терминалът е основният инструмент.

В заключение, въпреки че Firefox е по-разпространен сред Linux потребителите, Google Chrome е предпочетен заради своята универсалност, богат избор от теми и разширения и доказана стабилност. Това го прави най-подходящият браузър за конкретната конфигурация.

16. Заснемане на екрана: Gscreenshot

За нуждите на документиране, демонстрация на конфигурацията и създаване на визуално съдържание, изборът на инструмент за заснемане на екрана е също толкова важен, колкото и другите основни компоненти в една минималистична Linux система. В тази връзка е избран Gscreenshot – лек графичен инструмент за правене на снимки на целия екран, отделен прозорец или избрана зона. Програмата работи бързо и се интегрира добре в среда, изградена около i3wm и Thunar.

Gscreenshot е приложение, създадено с цел да предлага проста, но ефективна функционалност без излишни усложнения. Той не изисква тежки графични библиотеки и стартира мигновено, което го прави подходящ за системи с ниско натоварване на ресурсите. Интерфейсът му е изчистен и предлага само най-необходимите функции – избор на режим за снимане, забавяне при нужда и бърз достъп до мястото за съхранение на заснетите изображения. Това го прави особено удобен за среди, в които графичният интерфейс не е доминиращ, но е необходимо бързо и лесно визуално заснемане на работната повърхност или отделни прозорци.

Преди да бъде избран Gscreenshot, са разгледани и други инструменти като scrot, flameshot, shutter, spectacle и gnome-screenshot. Scrot е терминален инструмент с добра функционалност, но липсата на визуален интерфейс го прави по-малко интуитивен при избор на зони. Flameshot предлага разширени възможности за анотация, но изисква повече ресурси и предоставя повече функционалности, отколкото са необходими в този случай. Shutter и gnome-screenshot осигуряват богати графични възможности, но са обвързани с определени графични среди и зависят от библиотеки, които не са нужни в минималистичната конфигурация. Spectacle, като част от KDE, отпада по същите причини.

Gscreenshot осигурява добър баланс между лекота, бързина и функционалност. Не се нуждае от сложна конфигурация и работи стабилно с i3wm,

като може лесно да се извика чрез клавишна комбинация. Простотата му го прави подходящ за ежедневна употреба – както при архивиране на визуалната конфигурация, така и при създаване на примери за документация.

В заключение, Gscreenshot е предпочетен като инструмент за заснемане на екрана заради своята лекота, бързо действие и добра съвместимост с минималистична графична среда. Той изпълнява функцията си ефективно и без излишна сложност, напълно в синхрон с философията на системата.[34]

17. Преглед и задаване на тапет: Feh като минималистичен инструмент

В минималистична Linux среда, където се използва мениджър на прозорци като i3wm, липсва вграден инструмент за визуализиране на изображения или задаване на тапет на работния плот. Поради тази причина е избран Feh – лек и изключително гъвкав терминален инструмент за преглед на изображения, който често се използва и за задаване на фон в среди без пълноценен десктоп мениджър. Feh отговаря на изискванията за лекота, бързодействие и функционалност, което го прави подходящо допълнение към подобна система.

Основната причина за избора на Feh е свързана не само с възможността за преглед на изображения в отделен прозорец, но и с лесната му интеграция със стартиращи скриптове, позволяваща автоматично задаване на тапет при зареждане на системата – например чрез файл като `~/.config/i3/config`. Това е широко прилаган подход в среди, базирани на i3wm, Openbox и други леки window мениджъри. Програмата поддържа разнообразни режими на показване, включително цял екран, галерия и слайдшоу, както и директни команди за завъртане, мащабиране и навигация чрез клавиатурата.[35]

Като алтернативи са разгледани инструменти като sxiv, Viewnior, gpicview и nomacs. Sxiv също е лек и подходящ за терминална среда, но вече не се поддържа активно и не предлага функционалност за задаване на тапет. Viewnior и Gpicview са графични приложения, създадени за среди като LXDE или XFCE, но изискват повече зависимости и не се интегрират толкова естествено в среди без традиционен десктоп. Nomacs е по-функционален зрител на изображения, но е сравнително тежък и съдържа функции, които не са необходими в минималистична конфигурация.[36][37][38][39]

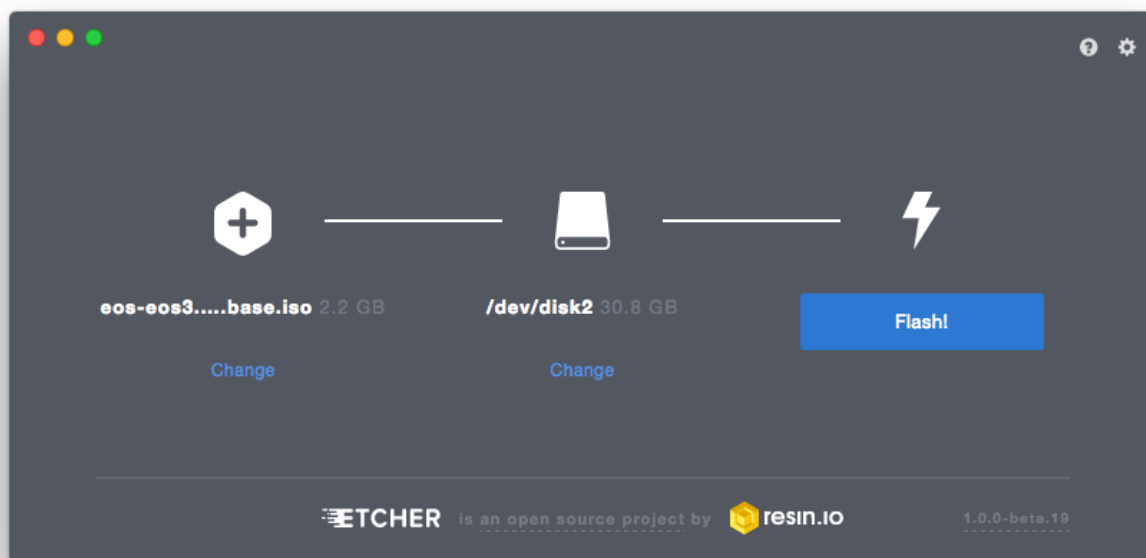
Feh се стартира мигновено, не изисква допълнителна графична среда и може да бъде управляван изцяло чрез команден ред или скриптове. Това го прави напълно съвместим с концепцията на минималистична Linux система, в която всеки компонент изпълнява ясно дефинирана роля, без излишна тежест и сложност. Освен това, Feh е добре документиран и широко използван сред потребители на window мениджъри, което допринася за стабилността и предвидимостта му като избор.

В заключение, Feh е предпочетен инструмент заради своята лекота, терминална ориентация и възможностите за автоматизирано задаване на тапет – качества, които го правят особено подходящ за среди, изградени върху i3wm и подобни конфигурации, в които минимализмът и контролът са водещи принципи.

Реализация на проекта

1. Подготовка на инсталационен носител

За да се създаде работна среда за операционната система (ОС), първата стъпка включва подготовката на USB флаш устройство, от което да се инсталира Arch Linux. За целта е необходимо да се изтегли ISO файлът, съдържащ дистрибуцията на операционната система. Официалният ISO файл може да бъде изтеглен от уебсайта на Arch Linux (archlinux.org). След изтегляне, ISO файлът трябва да бъде записан („изпечен“) на USB флаш устройство с помощта на специализиран софтуер. Един от най-популярните инструменти за тази задача е Balena Etcher.



Снимка 1 - използване на Balena Ether

Balena Etcher е безплатен и мултиплатформен софтуер, предназначен за флашване на образи на операционни системи върху USB и SD карти. Той е разработен с цел максимално опростяване на процеса по записване на ISO файлове и осигурява защита срещу записване върху грешни устройства. Програмата поддържа Windows, macOS и Linux и се отличава с интуитивен графичен интерфейс. Освен Balena Etcher, може да бъде използван и друг софтуер - Rufus, за да се постигне решението на първата стъпка.

Характеристика	Balena Etcher	Rufus
Поддържани ОС	Windows, MacOS, Linux	само Windows
Графичен интерфейс (GUI)	Да	Да
Поддръжка на ISO	Да	Да
Записване на USB	Да	Да
Проверка на целостта на данните	Да	Не
Скорост на записване	По-бавен	По-бърз

Поддръжка на множество файлови системи	Ограничена	Разширена
--	------------	-----------

Таблица 1 - Сравнение между Balena Etcher и Rufus

Balena Etcher е подходящ за потребители, които търсят лесен и сигурен начин за записване на ISO файлове, докато Rufus предлага по-голяма гъвкавост и допълнителни опции за напреднали потребители.

2. Стартиране на инсталационната среда

След успешното записване на ISO файла върху флаш устройството, следващата стъпка е да се рестартира компютърът и да се стартира системата от USB носител. Това обикновено изисква влизане в BIOS/UEFI менюто и задаване на USB устройството като първи приоритет в реда за зареждане. Тази стъпка е необходима, защото без нея компютърът се опитва да зареди операционната система от твърдия диск, а не от инсталационния носител. Задаването на USB като първи източник за зареждане гарантира, че се стартира правилната инсталационна среда.

След като системата се стартира от USB носител и се зарежда инсталационната среда на Arch Linux, се избира първата опция за зареждане на система от менюто, в което компютърът се boot-ва. Изборът на първата опция от boot менюто зарежда базовото ядро и минималната Linux среда, необходима за началната инсталация. Това е стандартният начин за влизане в живата (live) сесия на Arch Linux. След зареждането е необходимо първо да се актуализират основните пакети, което се извършва с командата:

```
pacman -Sy pacman
```

Тази команда актуализира базовата информация за наличните пакети в системата и конкретно пакета pacman, който е основният пакетен мениджър на Arch. Това е важно, защото стара версия на pacman може да доведе до грешки при по-нататъшната инсталация на софтуер. След актуализацията се инсталира помощният инструмент archinstall, който улеснява процеса на инсталация. Това се извършва с командата:

```
pacman -Sy archinstall
```

Инсталирането на archinstall предоставя полуавтоматизиран процес по инсталиране, който е много по-лесен за начинаещи потребители. Без него инсталацията би трябвало да се прави изцяло ръчно, чрез множество команди.

След изпълнение на тези стъпки, средата е готова за инсталация и персонализация на Arch Linux.

3. Инсталиране на Arch Linux

След подготовката и стартирането на инсталационната среда, основният процес по инсталиране на Arch Linux се извършва с помощта на вградения в дистрибуцията помощен инструмент `archinstall`. Той предлага полуавтоматизиран подход към инсталацията, като води потребителя през серия от конфигурационни менюта. Използването на `archinstall` значително улеснява инсталационния процес, особено за начинаещи, тъй като елиминира нуждата от ръчно въвеждане на всяка отделна команда. След зареждане на инсталационната среда, от командния ред се въвежда командата:

```
archinstall
```

Това стартира графичен текстов интерфейс, в който потребителят може да конфигурира ключови параметри на системата. Първата стъпка включва избора на огледала за изтегляне на софтуерни пакети. Препоръчително е да се избере "Bulgaria", когато инсталацията се извършва в рамките на страната, за да се осигури оптимална скорост на изтегляне. Изборът на близки огледала намалява времето за изтегляне и минимизира възможността от прекъсвания поради отдалечени сървъри.

В секцията `Disk configuration` се избира опцията `Use a best-effort default partition layout`, след което се посочва конкретният диск, върху който се извършва инсталацията. Важно е да се знае, че целият диск се форматира – всички налични данни се изтриват. Затова е необходимо предварително архивиране на важна информация. Препоръчително е и създаване на отделна директория `/home`, което осигурява по-добра организация на потребителските данни и улеснява бъдещото управление на системата. Това разделение е полезно при преинсталация на системата, тъй като позволява да се запазят личните файлове, без да се налага тяхното ръчно архивиране.

За зареждане на операционната система се използва `bootloader`. В тази конфигурация е избран `Limine` – модерен и лек зареждащ мениджър, поддържащ както BIOS, така и UEFI среди. `Limine` се отличава с висока скорост и лесна конфигурация, което го прави подходящ избор за персонализирани дистрибуции. Изборът на лек `bootloader` като `Limine` е особено подходящ при системи с по-ниски хардуерни ресурси или при търсене на бързо време за зареждане.

Hostname е името на машината в мрежата. В примера е зададено “MatriX”, но всеки потребител може да избере друго по желание. Името на машината служи за идентификация в локалната мрежа и в терминала, затова е добре да бъде кратко и запомнящо се.

На този етап се създава root парола – тя е необходима за административен достъп. Изборът на силна парола е от съществено значение за сигурността на системата. Без root парола системата остава уязвима, тъй като административни права могат да бъдат достъпени по подразбиране.

След това се добавя обикновен потребителски акаунт, например с име “zert”. Задават се парола и права за достъп. Използването на отделен потребителски акаунт вместо директна работа като root е добра практика за сигурност и стабилност на системата.

В менюто Profile се извършва избор на графична сесия и драйвери. В секцията type се избира Xorg – стандартната графична подсистема за UNIX-базирани системи. В секцията Driver се избира подходящ драйвер според производителя на видеокартата – nvidia, amdgpu, intel и др. Правилният избор на драйвер гарантира, че графичната карта работи с пълния си капацитет и няма да предизвиква проблеми с производителността или визуализацията. След изход и повторно влизане в менюто, се появява допълнителна подкатегория:

Greeter – избира се lυ, който е лек и минималистичен дисплей мениджър с текстов интерфейс. Този дисплей мениджър позволява бързо и леко влизане в графичната среда без ненужно натоварване на системата.

В секцията Audio се избира PulseAudio – популярен сървър за управление на звука с широки възможности за конфигурация. PulseAudio се използва широко заради съвместимостта си с повечето приложения и неговите възможности за контрол на множество аудио източници. Допълнителни настройки:

Kernels опцията може да бъде пропусната, тъй като по подразбиране се избира стабилното Linux ядро. В Additional Packages се изписва git като допълнителен пакет. Това е основен инструмент за управление на сорс код и изтегляне на файлове от GitHub. Network Configuration се избира Use NetworkManager – графичен инструмент за управление на мрежовите връзки, който предлага лесна конфигурация чрез GUI или CLI. На Timezone се задава коректната часова зона, например Europe/Sofia, за да бъде времето в системата синхронизирано правилно. В Additional Repositories се активира multilib, което

осигурява достъп до 32-битови библиотеки, нужни за някои специфични приложения или игри.

След като всички настройки са завършени, се избира Install от главното меню. Инсталацията протича автоматично и обикновено отнема от няколко до десетина минути. В края на процеса системата предлага избор дали да се направят допълнителни след инсталационни конфигурации. В този случай се избира No, което води до излизане от инсталационната среда и връщане към терминала. За да се стартира ново инсталираната система, се въвежда командата:

```
reboot
```

След рестартиране, ако всичко е конфигурирано правилно, Arch Linux се стартира с всички предварително зададени параметри.

4. Първо стартиране и начална конфигурация на системата

След успешното инсталиране на Arch Linux, системата се стартира и потребителят е посрещнат от графичния вход (greeter). Първата стъпка е да се избере вход в shell среда. В конзолата се въвеждат предварително зададеното потребителско име и съответната парола. Това осигурява достъп до основната терминална сесия, в която се изпълняват административни и конфигурационни команди.

4.1 Изтегляне на пакети с pacman

След влизане в системата чрез терминал, е необходимо да се инсталират основни допълнителни пакети, които осигуряват базова функционалност на средата. Това се извършва с командата:

```
sudo pacman -Sy base-devel nano file-roller neofetch btop pulsemixer alacritty  
thunar polybar rofi feh plymouth ttf-sourcecodepro-nerd
```

Тези пакети включват необходимите инструменти за разработка, редактиране на файлове, визуални компоненти и управление на системата. Например, nano е лек текстов редактор, neofetch показва системна информация, alacritty е терминал, а polybar и rofi са ключови компоненти за графичната среда.

4.2 Инсталиране на AUR помощник (helper)

Arch User Repository (AUR) е неофициално хранилище, поддържано от общността, което предоставя достъп до голямо разнообразие от допълнителни пакети, които не се намират в официалните репозитории на Arch. За да се инсталират такива пакети по-лесно, се използва помощник (AUR helper), който

автоматизира процеса на компилация и инсталация. Един от най-популярните помощници е paru. Инсталира се чрез следните команди:

```
git clone https://aur.archlinux.org/paru.git
cd paru/
makepkg -si --noconfirm
cd ..
rm -rf paru/
```

След успешната инсталация на paru, той се използва за инсталиране на AUR пакети по подобен начин на pacman. Това значително ускорява процеса по добавяне на нов софтуер, който не е наличен в официалните хранилища на Arch.

4.3 Инсталиране на допълнителни пакети чрез AUR помощник

С помощта на paru се инсталират следните пакети, които допълнително разширяват визуалната и функционална част на работната среда:

```
paru -Sy pod2man bibata-cursor-theme-bin gruvbox-gtk-theme-git gruvbox-
icon-theme-git google-chrome gsscreenshot plymouth-theme-loader-2-git i3-rounded-
border-patch-git i3-auto-tiling picom-simpleanimations-next-git
```

Тези пакети предоставят курсори, теми, икони, анимации и други визуални и функционални елементи, които допринасят за по-добър потребителски интерфейс. Google Chrome е добавен като основен браузър, а i3-auto-tiling и picom осигуряват управление на прозорците и визуални ефекти. След приключване на инсталацията, системата се рестартира с командата:

```
sudo shutdown now
```

Този рестарт е необходим, за да се приложат всички промени и да се зареди системата в пълнофункционален режим. При следващото зареждане на системата, във входящото меню (greeter) вече е налична и опцията за стартиране на графичната среда i3. След вход с потребителско име и парола, се стартира графичен интерфейс с помощта на клавишната комбинация Super+Enter (където Super е Windows бутонът, а на Mac — Command). Това потвърждава, че инсталираната графична среда функционира правилно и е готова за персонализация и работа.

5. Редактиране на файлове, които изискват привилегии

5.1 Редактиране на pacman.conf

Файлът на пакетния мениджър pacman, който се намира в /etc/pacman.conf, се редактира чрез командата:

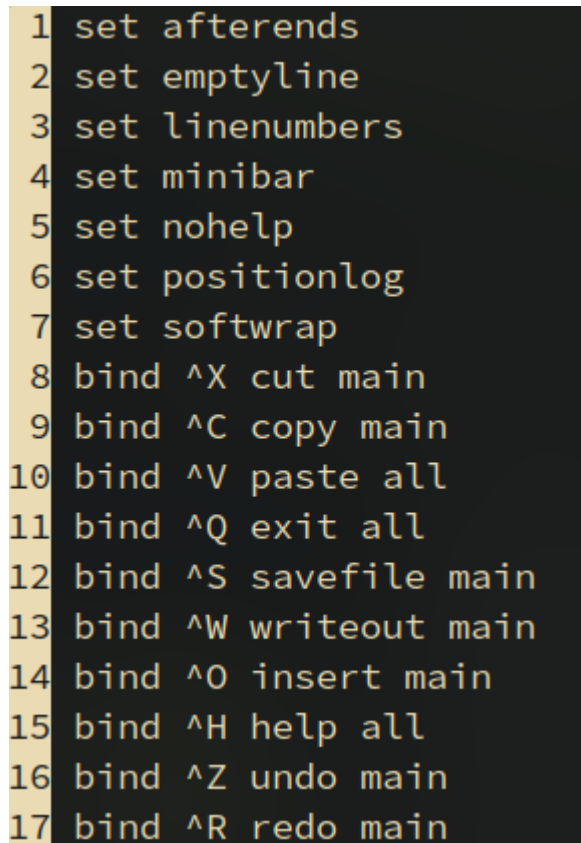
```
sudo nano /etc/pacman.conf
```

Маха се коментарът (#) от реда #Color, за да се активира цветният изход. Цветният изход прави работата в терминала по-прегледна и удобна, особено при списъци с множество пакети. Също така се проверява дали редът ParallelDownloads е активиран — това позволява едновременно изтегляне на повече от един пакет, което значително ускорява инсталацията и обновяването на системата.

5.2 Редактиране на nanorc

Редактира се файлът nanorc, като се добавят следните настройки за по-лесна и удобна работа с текстовия редактор nano. Файлът се редактира с командата:

```
sudo nano /etc/nanorc
```



```
1 set afterends
2 set emptyline
3 set linenumbers
4 set minibar
5 set nohelp
6 set positionlog
7 set softwrap
8 bind ^X cut main
9 bind ^C copy main
10 bind ^V paste all
11 bind ^Q exit all
12 bind ^S savefile main
13 bind ^W writeout main
14 bind ^O insert main
15 bind ^H help all
16 bind ^Z undo main
17 bind ^R redo main
```

Снимка 2 - Примерен конфигурационен файл на текстовия редактор nano

Тези настройки добавят функции като номериране на редовете, автоматично запомняне на позицията при отваряне на файл, включване на лента със статуси и удобни клавишни комбинации. Целта е nano да се превърне във функционален и лек заместител на по-тежки редактори, без да се прави компромис с удобството при работа.

5.3 Конфигуриране на Limine

Limine се конфигурира чрез редактиране на конфигурационния файл, който се намира в `/boot/limine.conf`. Целта е да се активира така нареченият `silent boot`, който премахва ненужната системна информация по време на стартиране. Това прави процеса на зареждане по-бърз, по-чист визуално и по-професионален за крайния потребител. За тази цел се правят две основни промени:

Стойността на `timeout` се променя от 5 на 0. Това означава, че менюто на зареждащия мениджър няма да изчаква, а зарежда автоматично избраната система.

Към параметъра `kernel_cmdline` се добавят следните флагове:

```
quiet  
loglevel=3  
systemd.show_status=auto  
rd.udev.log_level=3
```

Тези настройки минимизират показването на съобщения и логове при стартиране, без да се премахва напълно полезната информация. Това е подходящо както за обикновени потребители, така и за изграждане на системи с по-професионален и завършен вид.

6 Влизане в i3 сесия чрез ly и конфигурация на i3

След стартиране на графичната среда, следва конфигуриране на `i3 window manager` според предпочитанията на потребителя. Това се извършва чрез избор на графичната сесия `i3` още от входящото меню (`greeter ly`). След логване в системата, се отваря терминал с клавишната комбинация `Mod+Enter`. По подразбиране `Mod` е `Windows` клавишът на клавиатурата, а на `Mac` – `Command`. След отваряне на терминала се редактира конфигурационният файл на `i3`, който се намира в директорията `~/.config/i3/config`. Файлът се отваря с командата:

```
nano ~/.config/i3/config
```

Изтрива се съдържанието на файла с изключение на няколко “модула”, които се отнасят до базови операции като:

```
# Switch and move focused container to workspace n  
# Resize window (you can also use the mouse for that)  
# Exit i3 (logs you out of your X session)
```

Запазването на тези „модули“ гарантира, че основните действия продължават да работят дори при промяна на останалата конфигурация.

6.1 Местоположение и редакция на i3 конфигурационен файл

Първата съществена промяна е свързана с премахването на вградената лента (status bar) на i3 и нейната замяна с по-гъвкавата и визуално по-привлекателна polybar. За целта се премахват всички редове, свързани с bar в края на файла, и се добавят следните команди:

```
7 exec_always killall polybar
8 exec_always --no-startup-id bash ~/.config/polybar/launch_polybar.sh
```

Снимка 3 - Примерна конфигурация за стартиране на polybar във конфиг
файлът на i3

Тези редове стартират потребителския скрипт за Polybar и предотвратяват дублиране или конфликти при презареждане на прозоречния мениджър. Следващите редове, които се добавят в конфигурацията, служат за подобряване на визията и поведението на прозорците.

```
9 exec_always feh --bg-scale ~/.config/background/bg.png
10 exec_always --no-startup-id picom
11 exec_always --no-startup-id i3-auto-tiling
12
13 font pango:SauceCodeProNerdFont 14
14
15 gaps inner 15
16 gaps outer 5
17 default_border pixel 2
18 default_floating_border pixel 2
19
20 set $mod Mod4
21 set $smod Shift
```

Снимка 4 - Примерен конфигурационен файл за i3 който добавя подобрена
визия и поведение на прозорци

6.2 Навигация и манипулиране на прозорците

В тази секция се конфигурират клавишни комбинации, с които се управлява фокусът върху прозорците, тяхното размятане, преоразмеряване, максимизиране и други функции. Тези контроли правят работата в i3 изключително бърза и ефективна, защото елиминират нуждата от използване на мишка. С няколко натискания на клавиши потребителят мести, групира или подрежда прозорци според предпочитанията си, включително навигация между

работни пространства, преместване на прозорци между тях и промяна на режима от хоризонтално към вертикално подреждане.

```
27 ### Change window focus ###
28 bindsym $mod+Left      focus left
29 bindsym $mod+Down      focus down
30 bindsym $mod+Up         focus up
31 bindsym $mod+Right      focus right
32
33 ### Move focused window ###
34 bindsym $mod+$smod+Left move left
35 bindsym $mod+$smod+Down move down
36 bindsym $mod+$smod+Up   move up
37 bindsym $mod+$smod+Right move right
38
39 # System binds
40 bindsym $mod+$smod+q kill
41 bindsym $mod+$smod+r restart
42 bindsym $mod+f fullscreen toggle
43 bindsym $mod+h split v
44 bindsym $mod+v split h
45 bindsym $mod+$smod+f floating toggle
46 bindsym $mod+w layout tabbed
47 bindsym $mod+e layout toggle split
```

Снимка 5 - Конфигурационен файл за i3 които съдържа контролите за навигация в wm

6.3 Контроли за функционални приложения

За добавяне на бърз достъп до стартиране на приложения, команди, смяна на фокуса към далечни прозорци и изпълнение на действия без нуждата от терминал се използва Rofi. Добавят се и комбинации от клавиши за отваряне на терминал, смяна на език, заснемане на екрана, както и за манипулиране на звука от изходните устройства.

```

51 # Terminal
52 bindsym $mod+Return exec alacritty
53
54 # App launcher binds
55 bindsym $mod+d exec rofi -show drun -theme $HOME/.config/rofi/style.rasi
56 bindsym $mod+$smod+d exec rofi -show run -theme $HOME/.config/rofi/style.rasi
57 bindsym $mod+Tab exec rofi -show window -theme $HOME/.config/rofi/style.rasi
58
59 # Language change
60 bindsym $mod+slash exec ~/.config/lang_tog.sh
61
62 # Screenshot tool binds
63 bindsym $mod+$smod+s exec gsscreenshot -c -s -f ~/Screenshots/
64
65 # Volume manipulation
66 bindsym $mod+period exec pactl set-sink-volume @DEFAULT_SINK@ +5%
67 bindsym $mod+comma exec pactl set-sink-volume @DEFAULT_SINK@ -5%
68 bindsym $mod+$smod+period exec pactl set-source-volume @DEFAULT_SOURCE@ +5%
69 bindsym $mod+$smod+comma exec pactl set-source-volume @DEFAULT_SOURCE@ -5%
70 bindsym $mod+m exec pactl set-sink-mute @DEFAULT_SINK@ toggle
71 bindsym $mod+$smod+m exec pactl set-source-mute @DEFAULT_SOURCE@ toggle

```

Снимка 6 - i3 конфигурация за функционални приложения

Също така, за задаване на цветове, които се използват от i3, се включва палитра в началото на конфигурационния файл. Това улеснява смяната на цветовата схема и поддържането на визуална последователност с останалите компоненти на графичната среда.

```

139 #Colors
140 set $bg #282828
141 set $red #cc241d
142 set $green #98971a
143 set $yellow #d79921
144 set $blue #458588
145 set $purple #b16286
146 set $aqua #689d68
147 set $gray #a89984
148 set $darkgray #1d2021
149
150 # class          border      backgr      text        indicator   child_border
151 client.focused    $green      $green      $darkgray   $aqua       $aqua
152 #client.focused_inactive $darkgray  $darkgray   $yellow     $yellow     $yellow
153 client.unfocused  $darkgray  $darkgray   $yellow     $yellow     $gray
154 client.urgent     $red        $red        $white      $red        $red

```

Снимка 7 - Палитра от цветове за i3

7 Конфигурация на терминала Alacritty

Конфигурационният файл на Alacritty се намира в `~/.config/alacritty/alacritty.toml`. Той определя визуалния стил, цветовата палитра, клавишни комбинации и шрифт на терминала. Alacritty е един от най-бързите терминали, написан на езика Rust, и е предпочитан от потребители, които търсят

бързодействие и лекота. Въпреки че няма графичен интерфейс за настройка, конфигурационният му файл е изключително гъвкав и лесен за персонализиране. Примерна конфигурация може да включва:

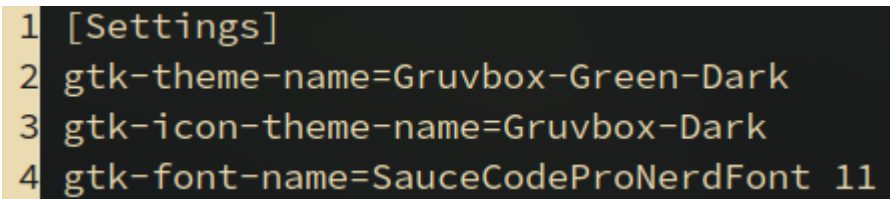
```
1  [env]
2  TERM = "xterm-256color"
3
4  [window]
5  opacity = 0.95
6
7  [font]
8  size = 14.0
9
10 [[keyboard.bindings]]
11 action = "Copy"
12 key = "C"
13 mods = "Control|Shift"
14
15 [[keyboard.bindings]]
16 action = "Paste"
17 key = "V"
18 mods = "Control|Shift"
19
20 [font.normal]
21 family = "SauceCodeProNerdFont"
22 style = "Regular"
23
24 [colors.selection]
25 background = "0x003300"
26 text = "0x00ff00"
27
28 [colors.primary]
29 background = "0x17191a"
30 bright_foreground = "0xffffffff"
31 dim_foreground = "0x9a9a9a"
32 foreground = "0xebdbb2"
33
34 [colors.normal]
35 black = "0x282828"
36 blue = "0x508a8d"
37 cyan = "0x689d6a"
38 green = "0xb5bd68"
39 magenta = "0xb16286"
40 red = "0xcc241d"
41 white = "0xa89984"
42 yellow = "0xd79921"
.config/alacritty/alacritty.toml *
```

Снимка 8 - Примерен конфигурационен файл за alacritty

Целта на тази конфигурация е да се осигури по-удобна работа с терминала, по-добра четимост, както и визуално съответствие с останалите части от графичната среда. Добре настроеният терминал значително подобрява продуктивността при работа с командния ред.

8. Теми и шрифтове за GTK приложения

Приложенията, използващи GTK, могат да бъдат конфигурирани чрез файл `settings.ini`, който се намира в директорията `~/.config/gtk-3.0/`. Този файл служи като централен конфигурационен документ за всички приложения, изградени върху GTK (като Thunar, File Roller и други), и позволява те да споделят един и същ външен вид. Използването на `settings.ini` дава възможност за уеднаквяване на визуалната среда, без нужда от ръчно конфигуриране на всяко отделно приложение. Това улеснява персонализирането и осигурява по-приятно и последователно потребителско изживяване. Примерна конфигурация в този файл може да изглежда така:



```
1 [Settings]
2 gtk-theme-name=Gruvbox-Green-Dark
3 gtk-icon-theme-name=Gruvbox-Dark
4 gtk-font-name=SauceCodeProNerdFont 11
```

Снимка 9 - GTK файл с примерено съдържание

Тази настройка осигурява унифициран външен вид на всички GTK-базирани приложения, като гарантира, че интерфейсът на системата изглежда завършен и визуално съгласуван.

9 Настройка на Polybar

9.1 Основен конфигурационен файл

Конфигурационният файл на Polybar се намира обикновено в `~/.config/polybar/config.ini`. Polybar служи като заместител на стандартната `i3` статус лента, предлагайки по-добър външен вид и значително повече възможности за персонализация. Файлът се състои от модули, като всеки модул е дефиниран с име, оградено в скоби `[]`. Този подход разделя конфигурацията по функционални секции (напр. звук, време, мрежа), което улеснява управлението и редакцията на отделните компоненти на лентата. Добавят се цветове, които служат като глобални стойности и се използват навсякъде в останалите модули, за да се гарантира визуална последователност.

```

1 [colors]
2 background = #282828
3 background-alt = #3c3836
4 foreground = #ebdbb2
5 primary = #a3be51
6 secondary = #83a598
7 alert = #fb4934
8 disabled = #928374

```

Снимка 10 - Модул палитра за polybar

Създава се лента с име по избор, например [bar/main]. Тази лента указва кои модули да се показват отляво, в центъра и отдясно. Използването на семпъл шрифт и символи гарантира добра четимост, дори на екрани с по-ниска резолюция.

```

14 [bar/Zer7]
15 width = 100%
16 height = 24pt
17 radius = 3
18
19 background = ${colors.background}
20 foreground = ${colors.foreground}
21
22 line-size = 3pt
23
24 border-size = 5pt
25 border-color = #00000000
26
27 padding-left = 0
28 padding-right = 0
29
30 module-margin = 1
31
32 separator = |
33 separator-foreground = ${colors.disabled};
34
35 font-0 = SouceCodeProNerdFont: size=14;2
36
37 modules-left = i3 program
38 modules-center = time
39 modules-right = pulseaudio online lang date

```

Снимка 11 - Bar модул за polybar

Модули, използвани за информация. Модул за език показва текущата подредба на клавиатурата. Модул за време показва ден, месец, година и час.

Модул за звук показва нивото на звука и дали е включен или изключен. Модул за интернет свързаност проверява дали системата има връзка с интернет. Тези модули дават важна системна информация с един поглед, което е основна цел на статус лентата.

```
64 [module/lang]
65 type = internal/xkeyboard
66 label-layout = Lang: %layout%
67
68 [module/time]
69 type = internal/date
70
71 ; Format
72 time = %H:%M:%S
73
74 ; Label
75 label = %time%
76
77 [module/date]
78 type = internal/date
79
80 ; Format
81 date = %A-%h: %d-%m-%y
82
83 ; Label
84 label = %date%
85
86 [module/pulseaudio]
87 type = internal/pulseaudio
88
89 ; Volume label
90 label-volume = VOL: %percentage%%
91
92 ; Muted label
93 label-muted = Muted
94
95 [module/online]
96 type = custom/script
97 exec = timeout 1 ping -c 1 8.8.8.8
98 interval = 5
99
100 format = <label>
101 format-foreground = ${colors.blue}
102 label = "NET: Connected"
103 format-padding = 0
104
105 format-fail = <label-fail>
106 format-fail-foreground = ${colors.red}
107 label-fail = "NET: Disconnected"
108 format-fail-padding = 0
```

Снимка 12 - Примерни модули за polybar

9.2 Скрипт за стартиране на Polybar

Създава се скрипт `~/.config/polybar/launch_polybar.sh`, който стартира Polybar на всички налични монитори. Това е необходимо, защото по подразбиране Polybar стартира само на един монитор. Ако системата разполага с повече от един екран, се налага ръчно да се зададе инстанция на лентата за всеки от тях. Скриптът автоматизира този процес и гарантира, че статус лентата се появява на всички екрани.

```
1 if type "xrandr"; then
2   for m in $(xrandr --query | grep " connected" | cut -d" " -f1); do
3     MONITOR=$m polybar --reload Zer7 &
4   done
5 else
6   polybar --reload Zer7 &
7 fi
```

Снимка 13 - Скрипт за стартиране на polybar

Примерно съдържание на скрипта проверява първо дали `xrandr` е наличен (това е инструментът, който извлича информация за свързаните екрани), след което стартира лентата `polybar` с името което сме задали в модула `[bar/]`, на всеки открит монитор. Така се избягва нуждата от ръчно стартиране на Polybar за всеки отделен дисплей.

След създаване, скриптът трябва да бъде направен изпълним с командата:

```
chmod +x ~/.config/polybar/launch_polybar.sh
```

10. Конфигурация на Rofi и други по малки приложения и полезни функции

10.1 Конфигурация на Rofi

За визуално подобрен Rofi се използва готова тема от хранилището на потребителя `adi1090x`, налично на: <https://github.com/adi1090x/rofi> Инсталира се тема `Type 4 / Style 1` с `Gruvbox` цвetoва схема. След изтегляне и преместване на файловете в `~/.config/rofi/`, се редактира стиливият файл (обикновено `style.rasi`), като се задава предпочитаният шрифт:

```
font: "SauceCodeProNerdFont 12";
```

Rofi е важен компонент в `i3`, който служи като `launcher` за програми и команди. Добре конфигуриран Rofi улеснява работата чрез клавиатура и визуално се вписва в останалата среда.

10.2 Създаване на команда SysMan

Създава се помощен скрипт SysMan, който автоматизира поддръжката на системата чрез актуализации, почистване и информационен изход. Скриптът се съхранява в ~/.config/SysMan:

```
1 #!/bin/bash
2 paru -Syu --noconfirm
3 sudo pacman -Rns $(pacman -Qdtq) --noconfirm
4 sudo pacman -Sc --noconfirm
5 clear
6 neofetch
```

Снимка 14 - Код в командата SysMan

Файлът се съхранява в ~/.config/SysMan и трябва да бъде направен изпълним чрез:

```
chmod +x ~/.config/SysMan
```

Тази команда позволява да се стартира целият процес по поддръжка само с едно действие, вместо всяка команда поотделно.

10.3 Включване на анимации (при наличие на подходящ хардуер)

Анимациите се управляват чрез модифицирана версия на композитора Picom. Тя включва вградени и оптимизирани анимации, които подобряват плавността на преходите между прозорци. Инсталацията се извършва чрез AUR:

```
paru -Sy picom-simpleanimations-next-git
```

Важно е компютърът да има поне базова видеокарта. Ако хардуерът не е достатъчно мощен, използването на анимации може да доведе до забавяне. При нужда, Picom може да бъде настроен в лек режим, като се промени променливата на редът backend = на "xrender".

10.4 Скрипт за смяна на езика

Този скрипт осигурява смяна на езика на клавиатурата при изпълнение. Ако е необходимо, могат да се добавят и други езици освен български, както и различни разновидности – традиционна, фонетична и други. В моя случай се използва само фонетичната подредба. Скриптът се записва в ~/.config/ с име lang_tog.sh

```
1 current_layout=$(setxkbmap -query | grep layout | awk '{print $2}')
2
3 if [ "$current_layout" = "us" ]; then
4     setxkbmap -layout bg -variant phonetic
5 else
6     setxkbmap -layout us
7 fi
```

Снимка 15 - Съдражние на скрипта lang_tog.sh

Файлът трябва да се направи изпълним с командата:

```
chmod +x ~/.config/lang_tog.sh
```

Това осигурява бърза смяна на езика с клавишна комбинация, без нужда от допълнителни програми.

10.5 Конфигурация на дребни приложения и елементи:

Курсор чрез .Xresources - за промяна на курсора се създава файл с име .Xresources в домашната директория. В него се задават настройки за стила и темата на курсора. Това осигурява съгласуван външен вид и в не-GTK приложения.

Конфигурация на bash - за допълнително удобство в терминала се инсталира oh-my-bash чрез следната команда:

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ohmybash/oh-my-bash/master/tools/install.sh)"
```

След това се редактира файлът .bashrc и се променя редът с темата на:

```
OSH_THEME="garo"
```

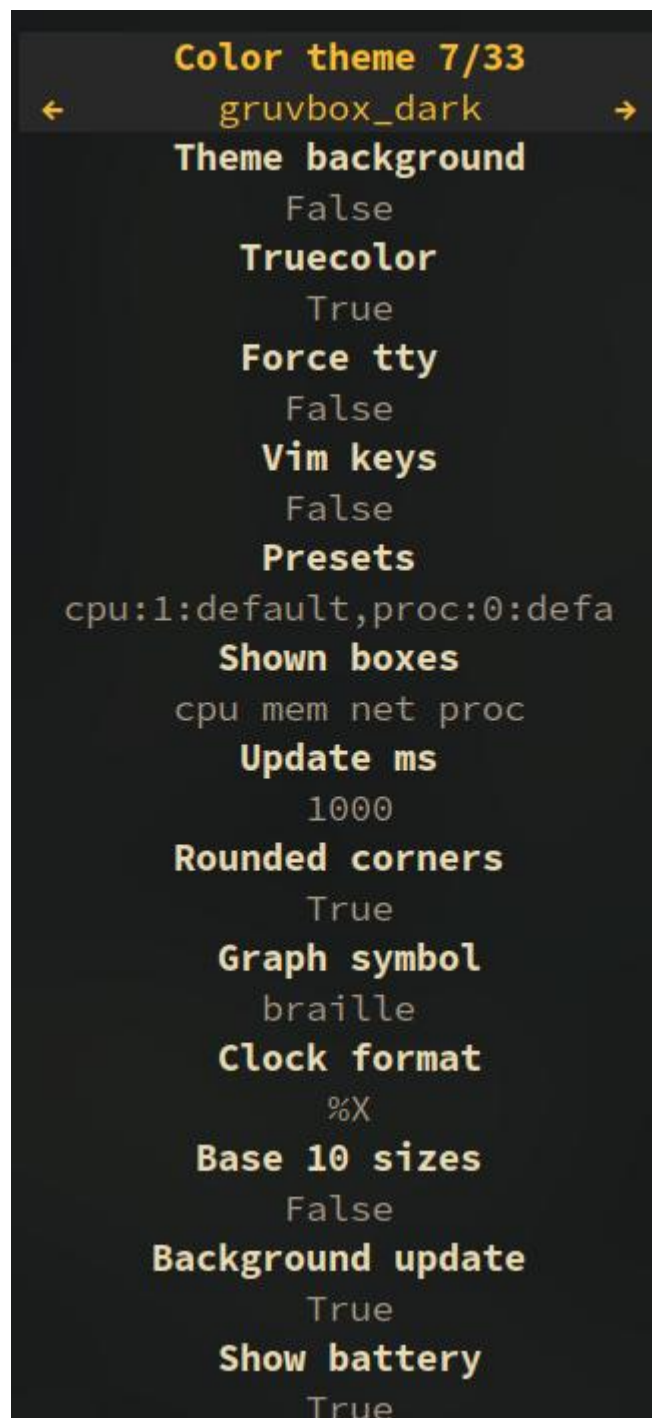
Това променя визията и цветовата схема на терминалния prompt.

Конфигурация на btop - Btop е графичен системен мениджър за терминал.

Стартира се с командата:

```
btop
```

След като се отвори, се натиска клавиш `m` за меню и се задават следните настройки:



Снимка 16 - Настройки на btop

Конфигурация на neofetch - Neofetch е инструмент за показване на системна информация в терминала. Конфигурацията се извършва чрез редакция на конфигурационния файл, като се оставят само следните редове активни: Uptime, Packages, Shell, GPU, CPU, Memory. Останалите редове се коментират със символ #. Също така се задава по-компактна ASCII графика чрез:

```
ascii_distro="arch_small"
```

Това прави изхода на Neofetch по-изчистен и подходящ за използване при всяко влизане в терминала.

11 Конфигурация на приложения

11.1 Настройка на файловия мениджър Thunar

За да се конфигурира файловият мениджър Thunar, първо се отваря, като това може да стане чрез въвеждане на името му в терминала или чрез търсене в Rofi. Thunar е лек, но мощен файлов мениджър, подходящ за среди като i3, тъй като не натоварва системата, но същевременно поддържа всички базови и разширени функции. След отваряне на приложението могат да се направят следните настройки. На менюто view се селектират - Show hidden files и statusbar и се изключват menubar и всички настройки от подменюто Side Pane.

11.2 Настройка на браузъра Google Chrome

След отваряне на браузъра и вход в Google акаунт, се подобрява потребителското изживяване чрез инсталиране на полезни разширения. Разширенията допълват функционалността на Chrome и позволяват визуално и практическо персонализиране на браузъра. Освен това се добавя и тема на браузъра, която допълва визуалната идентичност на работната среда.. Препоръчителни разширения и тема.

Gruvbox - тема за Google Chrome, създадена по Gruvbox цвetoва схема, която се използва в много терминали и програми с тъмен интерфейс. Тази подобрява визуалната последователност между терминала, стартера на приложения (Rofi), статус лентата (Polybar) и браузъра.

Dark Reader - автоматично прилага тъмен режим върху всички уебсайтове. Това е полезно за хора, които работят вечер или искат да намалят напрежението в очите. Не изисква допълнителна конфигурация.

New Tab Redirect - позволява задаване на собствена начална страница при отваряне на нов таб. Това е удобно за бърз достъп до най-често използваните линкове или персонализирани табла.

За персонализирана начална страница може да се използва следният адрес: <https://prettycoffee.github.io/yet-another-generic-startpage/#> След като се отвори началната страница и се персонализира (чрез иконата с двете стрелки в горния десен ъгъл)

Title

Gruvbox is the best Box

Image

https://e4p7c9i3.stackpathcdn.com/wp-content/uploads/2019/05/tumblr

☐ Display image

Font

Font size

font-size: 1.8rem

☒ Google fonts

Note: Fonts are automatically fetched from [google fonts](#) if enabled and possible. But you can also use local fonts.

Search

Searchbar placeholder

Search the web

Search engine

Custom search engine

google

e.g. https://search.brave.com/search?q={query}

☒ Search lookup +

Note: Bookmarks are automatically included in the lookup.

Theme

☒ Auto coloring

Background

Foreground

Primary

Secondary

#282828

#504945

#282828

#d4be98

Color presets

No selected

☐ Invert colors

☒ Post coloring

Note: Use **auto coloring** to set up your color theme and **post coloring** to adjust the details.

Surface

Max width

Border radius

max width: 1000px

13px border radius

Shadow count

Shadow offset

Shadow blur

2 shadow(s)

2px offset

3px blur

Снимка 17 - Настройки на началната страница

Следващата стъпка е отварянето на настройките на разширението New Tab Redirect, URL адресът от началната страница се поставя в съответното поле. След това се запазват настройките и всяко ново отворено табло зарежда новата начална страница. Това позволява напълно персонализиран стартов екран, който съответства на нуждите и стила на потребителя.

Заклучение

В настоящата дипломна работа изграждам напълно персонализирана Linux система, базирана на Arch Linux и съставена от внимателно подбран софтуер, съответстващ на принципите на минимализъм, ефективност и пълен контрол. Всеки компонент – от мениджъра на прозорци до системния мониторинг и графичните инструменти – е избран след анализ на алтернативите, техните предимства и недостатъци. Основният фокус е върху лекотата на системата, бързодействието и възможността за работа изцяло чрез клавиатура. Получената конфигурация предлага стабилна и визуално изчистена работна среда, подходяща както за продуктивна ежедневна употреба, така и за демонстрация на гъвкавостта на Linux екосистемата. В резултат системата успешно съчетава модерен подход със силен акцент върху практичността и контрола от страна на потребителя.

Источници

1. <https://bbs.archlinux.org/viewtopic.php?id=115942> - Последно посетен на 20.02.2025
2. <https://en.wikipedia.org/wiki/Ubuntu> - Последно посетен на 03.03.2025
3. <https://www.debian.org/intro/about> - Последно посетен на 14.01.2025
4. <https://docs.fedoraproject.org/en-US/fedora/latest/> - Последно посетен на 22.01.2025
5. http://wiki.manjaro.org/index.php/About_Manjaro - Последно посетен на 05.02.2025
6. https://en.wikipedia.org/wiki/Gentoo_Linux - Последно посетен на 10.02.2025
7. <https://voidlinux.org/#:~:text=Void%20Linux%20is%20an%20independent,have%20been%20written%20from%20scratch.> - Последно посетен на 17.03.2025
8. <https://linuxcommandlibrary.com/man/paru> - Последно посетен на 25.01.2025
9. <https://github.com/Jguer/yay> - Последно посетен на 29.03.2025
10. <https://github.com/actionless/pikaur> - Последно посетен на 07.02.2025
11. <https://wiki.archlinux.org/title/Xorg> - Последно посетен на 11.02.2025
12. <https://wayland.freedesktop.org/> - Последно посетен на 03.04.2025
13. https://wiki.archlinux.org/title/Window_manager - Последно посетен на 09.02.2025
14. <https://wiki.archlinux.org/title/Alacritty> - Последно посетен на 17.03.2025
15. <https://wiki.archlinux.org/title/Kitty> - Последно посетен на 12.01.2025
16. <https://wiki.archlinux.org/title/Rxvt-unicode> - Последно посетен на 25.02.2025
17. https://wiki.archlinux.org/title/List_of_applications/Utilities#File_managers - Последно посетен на 02.03.2025
18. <https://wiki.archlinux.org/title/Polybar> - Последно посетен на 13.03.2025
19. <https://i3wm.org/docs/userguide.html> - Последно посетен на 10.03.2025
20. <https://wiki.archlinux.org/title/Rofi> - Последно посетен на 20.01.2025

21. <https://wiki.archlinux.org/title/Dmenu> - Последно посетен на 28.02.2025
22. <https://wiki.archlinux.org/title/Picom> - Последно посетен на 17.02.2025
23. <https://archlinux.org/packages/extra/any/pulsemixer/> - Последно посетен на 22.03.2025
24. https://wiki.archlinux.org/title/Advanced_Linux_Sound_Architecture - Последно посетен на 04.03.2025
25. <https://www.freedesktop.org/wiki/Software/PulseAudio/> - Последно посетен на 15.03.2025
26. <https://htop.dev/> - Последно посетен на 08.01.2025
27. <https://github.com/aristocratos/btop> - Последно посетен на 06.04.2025
28. <https://github.com/cjbassi/gotop> - Последно посетен на 12.02.2025
29. <https://phoenixnap.com/kb/best-linux-text-editors-for-coding> - Последно посетен на 19.03.2025
30. https://en.wikipedia.org/wiki/File_Roller - Последно посетен на 23.01.2025
31. <https://en.wikipedia.org/wiki/Xarchiver> - Последно посетен на 14.02.2025
32. <https://wiki.mate-desktop.org/mate-desktop/applications/engrampa/> - Последно посетен на 16.02.2025
33. <https://apps.kde.org/ark/> - Последно посетен на 10.01.2025
34. https://wiki.archlinux.org/title/Screen_capture - Последно посетен на 11.03.2025
35. <https://feh.finalrewind.org/> - Последно посетен на 09.01.2025
36. <https://wiki.archlinux.org/title/Sxiv> - Последно посетен на 07.01.2025
37. <https://github.com/hellosiyan/Viewnior> - Последно посетен на 25.03.2025
38. <https://lxde.sourceforge.net/gpicview/> - Последно посетен на 13.01.2025
39. <https://nomacs.org/docs/documentation/features/> - Последно посетен на 01.03.2025